



Thiết kế P3 — Feedback Intelligence (Vòng lặp cải thiện chất lượng RAG)



Mục tiêu: Biến dữ liệu like/dislike từ chỗ "chỉ ghi nhận" thành một **vòng lặp cải thiện chất lượng** thực thụ — phát hiện câu trả lời sai, sửa, đem trả lời lại, và tránh để feedback cũ làm hiểu lầm metadata mới. Trang này là bản thiết kế để duyệt **trước khi code**.

1. Hiện trạng & lý do cần P3

- **Bug mapping like/dislike** (👍 bị ghi thành dislike, 👎 thành like) — đã sửa trong bản mới nhất.
- Sau khi rà toàn bộ `rag/` và `ingestion/`: **dữ liệu feedback / golden set hiện KHÔNG được dùng ở bất kỳ đâu trong luồng trả lời**. Like/dislike chỉ được ghi vào `LichSuChat.DanhGia` và (khi dislike) tạo dòng trong `FeedbackReview` để người duyệt đọc thủ công.
- Vì vậy vòng lặp "feedback → cải thiện AI" **chưa tồn tại, cần xây mới**.

2. Nguyên tắc thiết kế (đọc kỹ trước)

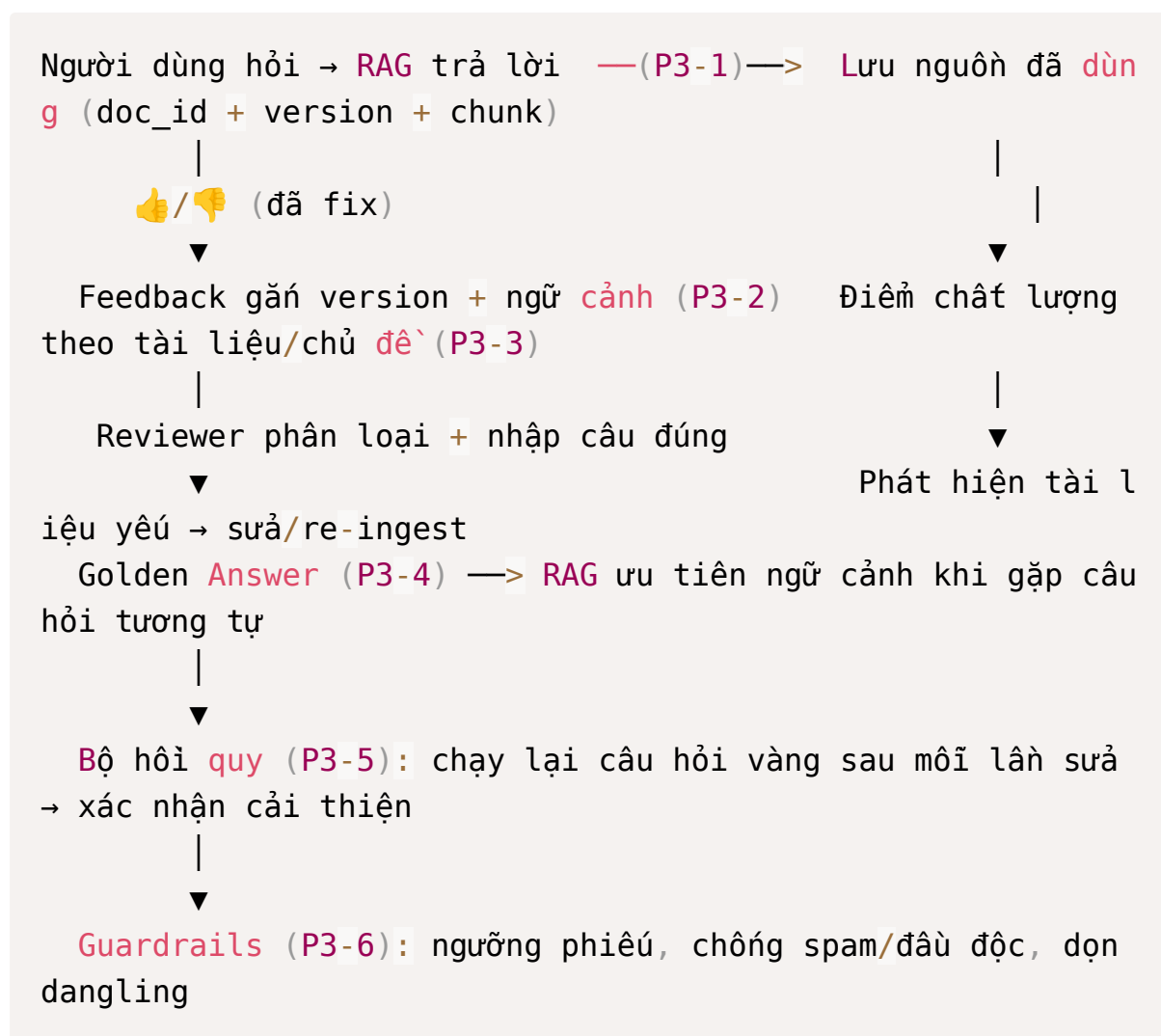


Hệ thống dùng LLM gọi qua API (gpt-5.4). **Không fine-tune/huấn luyện lại model** — không thực tế và tốn kém. Cải thiện được thực hiện qua **RAG + golden set + quy trình duyệt**, không phải "train" model.

- **Like ≠ đúng:** like chỉ là *tín hiệu*, người dùng có thể like vì "nghe hợp lý". Không bao giờ tự động coi like là chân lý.
- **Reviewer là công kiểm soát:** chỉ reviewer/admin mới được đẩy nội dung vào golden set.

- **Feedback luôn gắn (version tài liệu + ngữ cảnh truy xuất)**, không gộp toàn cục.
- **Có trọng số & giảm dần theo thời gian**: phiếu của chuyên gia nặng hơn; feedback cũ nhẹ dần.

3. Kiến trúc vòng lặp (tổng quan)



Khi một tài liệu được **re-ingest / superseded**, feedback cũ gắn version cũ được **tự đánh dấu "stale"** → không còn tính vào điểm chất lượng của bản mới (giải quyết trực tiếp lo ngại của bạn).

4. Mô hình dữ liệu mới (đề xuất)

Bảng / Cột	Vai trò	Mục P3
AnswerSource (ChatID, DocID, VersionNo, ChunkID, Score, Rank)	Lưu các chunk/tài liệu RAG đã dùng cho từng	P3-1

Bảng / Cột	Vai trò	Mục P3
	câu trả lời	
FeedbackReview • DocVersionNo , ContextHash , Department , Site , IsStale , ResolvedByDocID , ResolvedAt	Gắn feedback với version + ngữ cảnh; cờ stale	P3-2
GoldenAnswer (QuestionNorm, Answer, SourceDocID, VersionNo, Status, Weight, CreatedBy, CreatedAt)	Câu trả lời chuẩn để RAG ưu tiên	P3-4
DocQualityScore (DocID, Likes, Dislikes, WeightedScore, LastComputed) — <i>có thể là bảng tổng hợp hoặc view</i>	Điểm chất lượng theo tài liệu	P3-3
RegressionQuestion (QID, QuestionNorm, ExpectedDocID, ExpectedKeywords, Active) + RegressionRun (RunID, QID, Passed, AnswerSnapshot, RunAt)	Bộ hồi quy & lịch sử chạy	P3-5

5. Chi tiết từng mục

<details><summary>P3-1 — Truy vết nguồn (Source Attribution)</summary></details>

- **Mục tiêu:** Biết chính xác *tài liệu/chunk* nào đã tạo ra câu trả lời, để khi bị dislike còn biết sửa ở đâu.
- **Schema:** thêm bảng **AnswerSource**.
- **Code:** trong luồng RAG (**rag/service.py** / dispatch), sau khi lấy chunk → trả về danh sách (**doc_id**, **version_no**, **chunk_id**, **score**); **save_chat_history** lưu kèm vào **AnswerSource**.
- **Nghiệm thu:** mỗi câu trả lời có ≥1 dòng **AnswerSource**; mở 1 ChatID thấy đúng tài liệu nguồn.

</details>

<details><summary>P3-2 — Feedback gắn version + tự đánh dấu stale</summary></details>

- **Mục tiêu:** Tránh "cờ dislike trên metadata cũ" làm bản cập nhật mới bị hiểu nhầm là sai.
- **Schema:** **FeedbackReview** thêm **DocVersionNo**, **ContextHash**, **Department**, **Site**, **IsStale**, **ResolvedByDocID**, **ResolvedAt**.

- **Logic:** khi tạo feedback, ghi version + phòng ban + site của ngữ cảnh. Khi một DocID được **re-ingest/superseded** (đã có `SupersedesDocID` , `IsCurrent`), chạy bước **đánh dấu** `IsStale=1` cho mọi feedback gắn version cũ của tài liệu đó.
- **Nghiệm thu:** sửa metadata + re-ingest 1 tài liệu → feedback dislike cũ của nó chuyển `IsStale=1` , biến mất khỏi điểm chất lượng bản mới; có thể đưa vào hàng "cần test lại".

</details>

<details><summary>P3-3 — Điểm chất lượng theo tài liệu/chủ đề</summary>

- **Mục tiêu:** Tổng hợp like/dislike để lộ ra tài liệu/chủ đề yếu.
- **Logic:** điểm = $\Sigma(\text{phiếu} \times \text{trọng_số_vai_trò} \times \text{hệ_số_giảm_thời_gian})$, **bỏ qua** `IsStale=1` . Trọng số: admin/reviewer > uploader > viewer.
- **UI:** thêm bảng xếp hạng "tài liệu bị dislike nhiều" ở trang Báo cáo sử dụng / Dashboard.
- **Nghiệm thu:** tài liệu nhiều dislike (chưa stale) lên đầu bảng; feedback stale không tính.

</details>

<details><summary>P3-4 — Golden Answer cho RAG</summary>

- **Mục tiêu:** "Đem ra trả lời lại" — câu trả lời đúng đã duyệt được RAG ưu tiên khi gặp câu hỏi tương tự.
- **Schema:** bảng `GoldenAnswer` ; khi reviewer lưu `CorrectAnswer` + duyệt → tạo bản ghi golden (chuẩn hóa câu hỏi).
- **Code:** trước khi gọi LLM, tra `GoldenAnswer` theo câu hỏi chuẩn hóa (+ ngữ cảnh phòng ban/quyền); nếu khớp → **chèn làm ngữ cảnh ưu tiên** (không hard-code trả lời máy móc, vẫn để LLM diễn đạt nhưng bám golden).
- **Nghiệm thu:** câu hỏi từng bị dislike + đã có golden → lần hỏi sau trả lời đúng theo golden.

</details>

<details><summary>P3-5 — Bộ hồi quy (Regression Suite)</summary>

- **Mục tiêu:** Sau mỗi lần sửa/ingest, chạy lại tập câu hỏi vàng để xác nhận đã cải thiện, không gây hồi quy.
- **Schema:** `RegressionQuestion` , `RegressionRun` .
- **UI:** nút "Chạy hồi quy" (admin) → chạy batch, hiện Pass/Fail + so với lần trước.
- **Nghiệm thu:** chạy được tập câu hỏi, ra báo cáo Pass/Fail; câu đã sửa chuyển từ Fail → Pass.

</details>

<details><summary>P3-6 — Guardrails (chống nhiễu/đầu độc)</summary></details>

- **Ngưỡng phiếu:** chỉ hành động khi đủ số phiếu tối thiểu, không theo 1 phiếu.
- **Chống spam 1 người:** giới hạn 1 phiếu/người/câu trả lời (key theo Username + ChatID).
- **Chống đầu độc:** chỉ admin/reviewer được đẩy golden set; trọng số theo vai trò.
- **Dọn dangling:** ràng buộc/dọn `CorrectSourceDocID` , `ResolvedByDocID` khi tài liệu bị xóa.
- **Gộp câu trùng:** chuẩn hóa (bỏ dấu, lowercase) khi gộp thống kê/golden.

</details>

6. Bản đồ 8 tình huống rủi ro → giải pháp

#	Tình huống	Giải pháp (mục)
1	Like không = đúng (lịch sử, "nghe hợp lý")	Reviewer là cổng; không auto-train (P3-4, P3-6)
2	Đúng hôm nay, sai ngày mai (spec đổi)	Feedback gắn version + stale (P3-2)
3	Cùng câu hỏi, khác phòng ban/site/quyền	Feedback gắn ngữ cảnh truy xuất (P3-2, P3-4)
4	Dislike do câu hỏi mơ hồ, không phải lỗi bot	Phân loại <code>FailureType</code> (đã có) bắt buộc khi duyệt
5	Mâu thuẫn: vừa like vừa dislike	Ngưỡng tổng hợp (P3-3, P3-6)

#	Tình huống	Giải pháp (mục)
6	Tài liệu bị xóa/archive nhưng feedback còn trở	Dọn dangling reference (P3-6)
7	Đầu đọc golden set / user quyền thấp lũng đoạn	Trọng số vai trò + cổng reviewer (P3-3, P3-6)
8	Câu hỏi trùng khác cách viết	Chuẩn hóa khi gộp (P3-6)

7. Kế hoạch triển khai & thứ tự

1. **P3-1** (truy vết nguồn) — nền tảng, các mục sau phụ thuộc.
2. **P3-2** (version + stale) — giải quyết lo ngại chính.
3. **P3-3** (điểm chất lượng) — đọc dữ liệu P3-1/P3-2.
4. **P3-4** (golden answer) — bắt đầu cải thiện trả lời.
5. **P3-5** (hỏi quy) — kiểm chứng.
6. **P3-6** (guardrails) — siết an toàn, đan xen suốt quá trình.

Mỗi mục: **1 migration idempotent** + sửa code + tài liệu hướng dẫn + tiêu chí nghiệm thu, làm tuần tự rồi gửi bản code như các P trước.

8. Rủi ro cần lưu ý khi làm

- Truy vết nguồn (P3-1) yêu cầu RAG trả về metadata chunk — cần kiểm tra `rag/service.py` có sẵn `doc_id / version` trong payload Qdrant (đã thấy có `doc_id`, `version_no`).
- Golden answer (P3-4) phải tránh "đóng băng" câu trả lời sai: golden cũng gắn version, stale theo P3-2.
- Hiệu năng: tính điểm chất lượng nên chạy nền/định kỳ, không tính realtime mỗi lần mở dashboard.



Bước tiếp theo: Bạn duyệt bản thiết kế này. Nếu OK, mình bắt đầu code tuần tự từ **P3-1**, mỗi mục xong sẽ gửi bản code + hướng dẫn + tiêu chí test như các giai đoạn trước.